Section 3.3: The Forward Pass

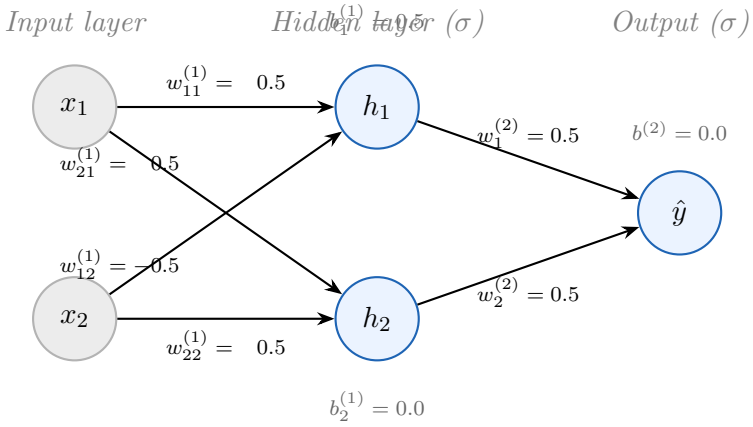
Computing a Prediction Step by Step

AI Class — Lecture Notes [Student Copy]

---

1 The Network Architecture

We will work with a **two-layer feedforward network** (one hidden layer) designed to solve XOR:



The network has the following parameters:

$$\mathbf{W}^{(1)} = \begin{bmatrix} 0.5 & -0.5 \\ 0.5 & 0.5 \end{bmatrix}, \quad \mathbf{b}^{(1)} = \begin{bmatrix} 0.5 \\ 0.0 \end{bmatrix}, \quad \mathbf{W}^{(2)} = \begin{bmatrix} 0.5 & 0.5 \end{bmatrix}, \quad b^{(2)} = 0.0.$$

Row i of $\mathbf{W}^{(1)}$ holds the weights *feeding into* hidden unit i. These weights are **not yet trained** — we will start gradient descent in Section 3.4.

2 Notation and the Sigmoid Function

Each neuron in the hidden and output layers uses the **sigmoid** activation:

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \qquad \sigma(z) \in (0, 1).$$

The computation inside every neuron follows two sub-steps:

1. **Pre-activation** z: a weighted sum of inputs plus bias.
2. **Activation** $h = \sigma(z)$: squeeze z into $(0, 1)$.

Since e^{-z} is not convenient to compute by hand, use the reference table below throughout this section and Section 3.4:

z	$\sigma(z)$	$\sigma'(z) = \sigma(z)\big(1-\sigma(z)\big)$
-1.0	0.269	0.197
-0.5	0.378	0.235
0.0	0.500	0.250
0.5	0.622	0.235
1.0	0.731	0.197

The derivative column $\sigma'(z)$ is not needed yet — it will be used in Section 3.4 (Back-propagation). It is included here so that both tables are in the same place.

3 The Forward Pass

We trace the network with the XOR input $\mathbf{x} = (x_1, x_2) = (0, 1)$, whose target output is $y = 1$.

Step 1 Input → Hidden pre-activations

Compute the weighted sum entering each hidden neuron:

$$z_1^{(1)} = w_{11}^{(1)}\, x_1 + w_{12}^{(1)}\, x_2 + b_1^{(1)} = (0.5)(0) + (-0.5)(1) + 0.5 = \_\_\_\_\_$$

$$z_2^{(1)} = w_{21}^{(1)}\, x_1 + w_{22}^{(1)}\, x_2 + b_2^{(1)} = (0.5)(0) + (0.5)(1) + 0.0 = \_\_\_\_\_$$

Step 2 Hidden activations

Apply the sigmoid to each pre-activation (use the reference table):

$$h_1 = \sigma\Big(z_1^{(1)}\Big) = \sigma(0.0) = \_\_\_\_\_$$

$$h_2 = \sigma\Big(z_2^{(1)}\Big) = \sigma(0.5) = \_\_\_\_\_$$

Step 3 Hidden → Output pre-activation

The output neuron collects the two hidden activations:

$$z^{(2)} = w_1^{(2)}\, h_1 + w_2^{(2)}\, h_2 + b^{(2)} = (0.5)(0.500) + (0.5)(0.622) + 0.0 = 0.250 + 0.311 = \_\_\_\_\_$$

Step 4 Output activation

$$\hat{y} = \sigma\left(z^{(2)}\right) = \sigma(0.561)$$

The value 0.561 is not in the reference table, but it lies between 0.5 and 1.0, so $\sigma(0.561)$ lies between 0.622 and 0.731. Interpolating (or using a calculator):

$$\hat{y} \approx \_\_\_\_\_$$

4 Computing the Loss

The target for input $(0, 1)$ is $y = 1$. Using Mean Squared Error:

$$J = (y - \hat{y})^2 = (1 - 0.637)^2 = (0.363)^2 \approx \_\_\_\_\_$$

The network is wrong: it outputs 0.637 when it should output 1. The loss $J = 0.132$ tells us how far off we are.

A perfect prediction ($\hat{y} = 1$) would give $J = 0$. A blind guess of $\hat{y} = 0.5$ would give $J = (0.5)^2 = 0.25$. Our untrained network ($J = 0.132$) happens to be a bit better than random — but only because we are looking at a single input.

5 Summary: What the Forward Pass Gives Us

Quantity	Symbol	Value
Input	$(x_1,\ x_2)$	$(0,\ 1)$
Target	y	1
Hidden pre-activation 1	$z_1^{(1)}$	0.0
Hidden pre-activation 2	$z_2^{(1)}$	0.5
Hidden activation 1	h_1	0.500
Hidden activation 2	h_2	0.622
Output pre-activation	$z^{(2)}$	0.561
Prediction	$\hat{y}$	0.637
Loss	J	0.132

These numbers are the starting point for Section 3.4. Backpropagation will work *backwards* through this same network to compute $\partial J/\partial w$ for every weight — which is exactly what gradient descent needs to take an update step.